

Sistemas Operativos 2026



Clase 7 U3 Interbloqueos (Deadlocks) I

Ing. Juan Manuel Calvo

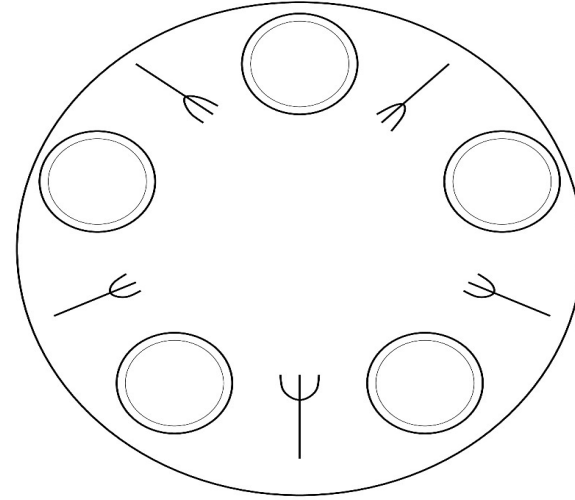
Problemas clásicos de sincronización

- Inventados para demostrar primitivas de sincronización
- No necesariamente análogos a problemas del mundo real
- Ilustran principios que deberían utilizar las soluciones reales



Los filósofos cenando

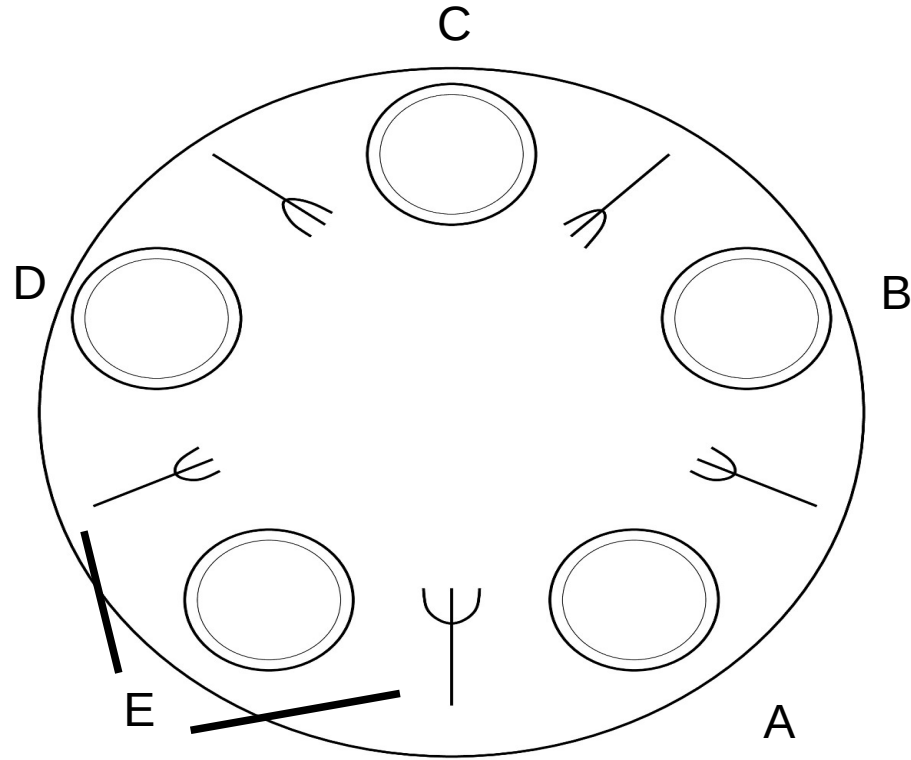
- Cinco filósofos sentados alrededor de una mesa
- Cada uno tiene un plato de espaguetis
- Hay tenedores entre cada par de platos
- Los filósofos necesitan dos tenedores para comer



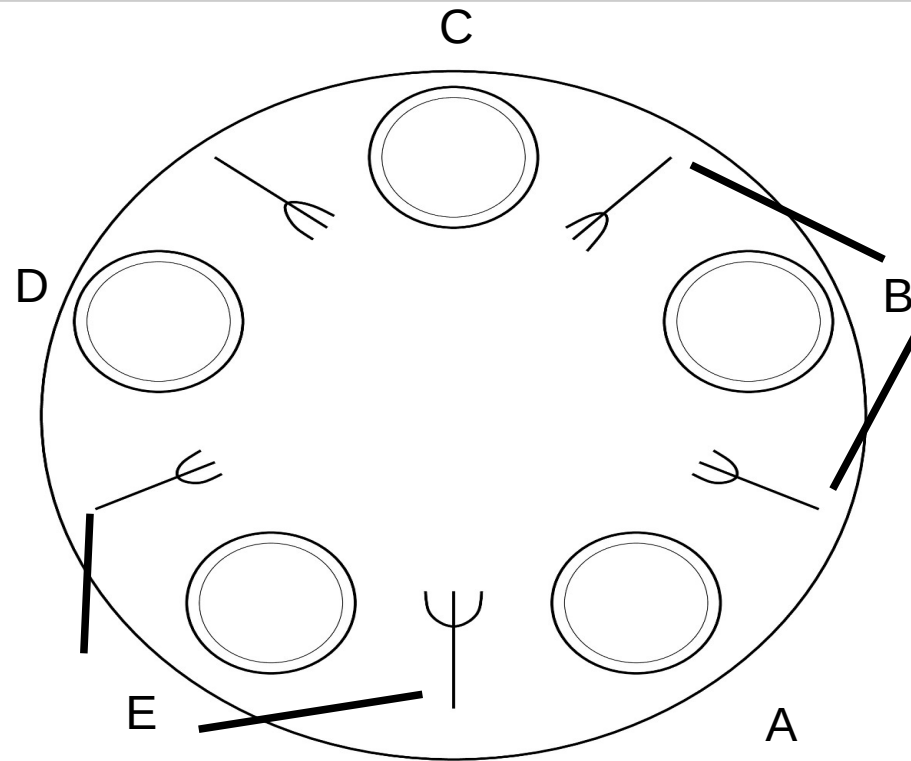
Pensar y comer

- Los filósofos se pasan la vida comiendo espaguetis o pensando.
- Cuando piensan, no necesitan ningún utensilio.
- Para comer, necesitan tomar tanto el tenedor de la izquierda como el de la derecha.

Eve está comiendo

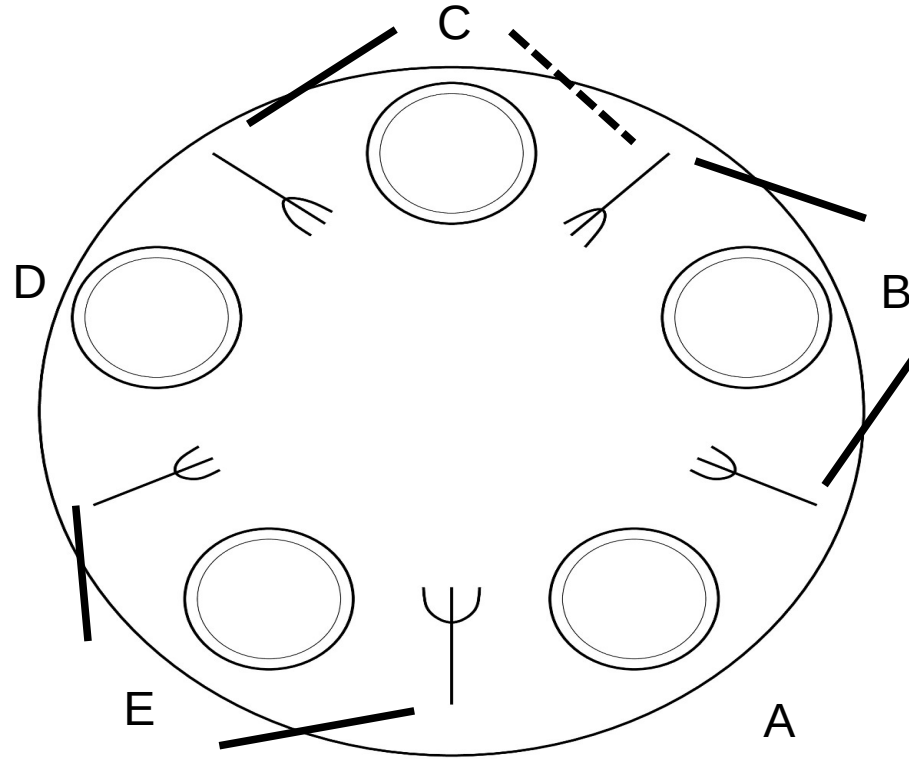


Eve y Bob están comiendo



UNIVERSIDAD
DEL CEMA
UCEMA

Pero Carol está bloqueada



El problema

- Encontrar un algoritmo que permita que todos coman.
- Es necesario evitar los interbloqueos.

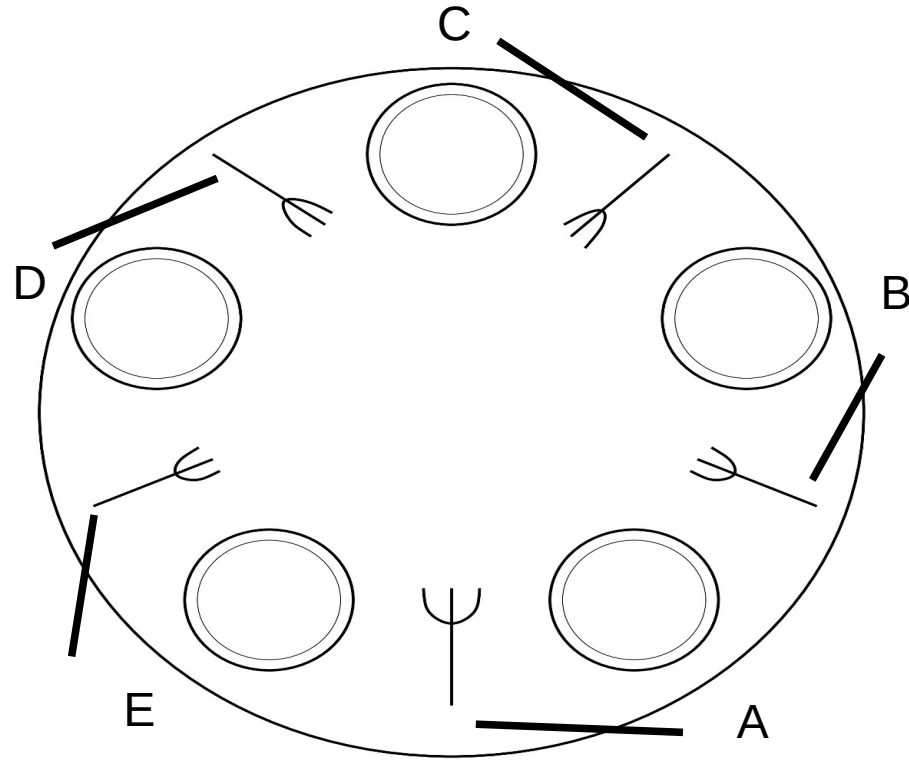


Una mala solución

```
void philosopher(int i)
{
    while(TRUE) {
        think();
        take_fork(i);
        take_fork((i+1) % N);
        eat();
        put_fork(i);
        put_fork((i+1) % N);
    }
}
```

Ver phil1.c

Si todos toman un tenedor ...



Reparemos la solución

- En lugar de tomar el tenedor $((i+1) \% N)$, compruebe primero si está disponible.
 - (Por supuesto, esta comprobación y la captura se realizan dentro de una región crítica).
- Si no está disponible, espera un rato y vuelve a intentarlo.
- Sigue sin funcionar. ¿Qué pasa si todos intentan el tenedor izquierda, esperan, vuelven a intentarlo simultáneamente, etc.?
- Todos pueden correr, pero nadie avanza: *inanición*.

Aleatorización

- Supongamos que los filósofos duermen durante un tiempo aleatorio si no consiguen el segundo tenedor.
- Normalmente, eso funcionará bien.
- Pero algunas aplicaciones requieren una solución ***garantizada***.

Ver [phil2.c](#)



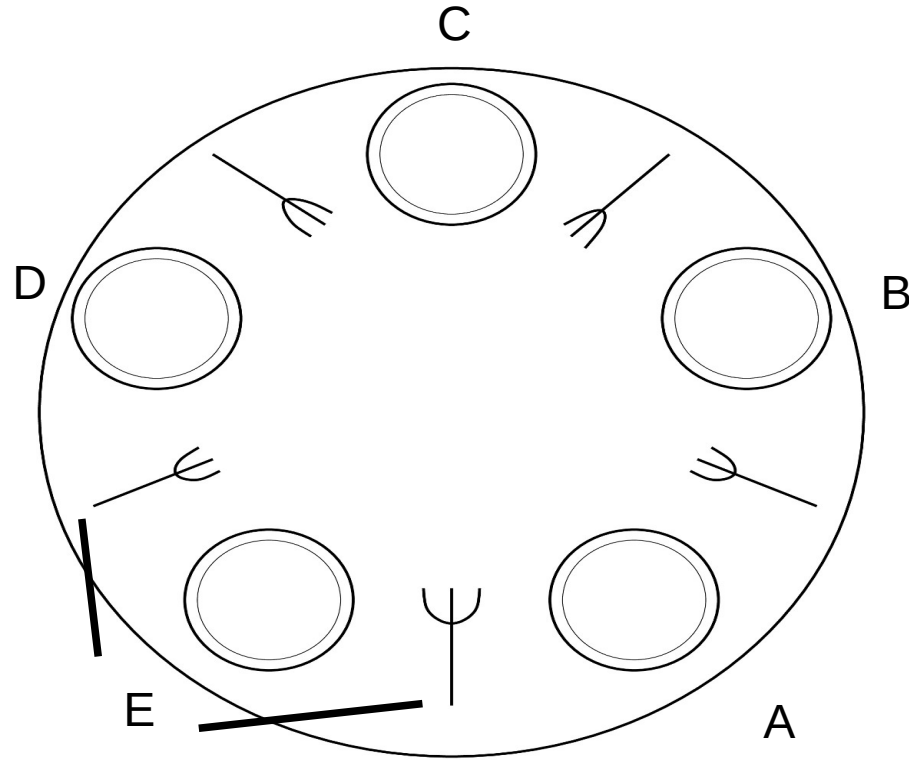
Una solución simple y garantizada

- Antes de tomar un tenedor, un filósofo bloquea un mutex.
- El filósofo puede entonces tomar dos tenedores sin interferencias.
- Al terminar de comer, se vuelven a colocar los tenedores y se libera el mutex.

¿Qué está mal?

- Tal como se presenta, solo un filósofo puede comer a la vez.
- Modifiquemos el algoritmo: supongamos que el filósofo — digamos, Eve — libera el mutex antes de comer.
- ¿Qué sucede ahora? ¿Resuelve esto el problema?

Eve está comiendo



No

- Alice, Bob, Carol y Dave ahora compiten por los otros tres tenedores.
- Volvemos al mismo problema: ¿cómo lo resolvemos de forma justa?

Principios de una buena solución

- Bloquear un mutex al intentar tomar los tenedores.
 - Si ambos están disponibles, comer.
 - Si uno no está disponible, marcarse con hambre y dormir.
- Al liberar los tenedores, comprobar si los vecinos tienen hambre; si es así, despertarlos.

Ver `phil3.cpp`



Por qué esto funciona: una aproximación de una demostración

- Supongamos que Eve está comiendo
- Alice y Dave no pueden comer; como máximo, uno de Bob y Carol pueden comer
- Supongamos que Bob y Carol están pensando. Tanto Alice como Dave se despertarán cuando Eva termine, por lo que obviamente todo avanzará. Por lo tanto, supongamos que Carol también está comiendo.
- Todos los demás que quieran comer están bloqueados
- Cuando Eva termina, Alice y Dave se despiertan. Dave sigue bloqueado, pero Alicia puede correr
- Cuando Carol termina, Bob puede correr
- Nota: Se debe asumir que comer es una actividad finita y que hay una cola en cada semáforo

Lectores y escritores

- Supongamos que tenemos una base de datos.
- Cualquier número de procesos puede leer simultáneamente, pero solo un proceso puede escribir a la vez.
- Nadie puede leer mientras se escribe.

Bosquejo de la solución

- Lleva un registro de cuántos lectores hay (usa un mutex).
- Cuando llegue el primer lector, bloquea el mutex de la base de datos.
- Los lectores posteriores se cuentan.
- Cuando todos los lectores se hayan ido, libera el mutex de la base de datos.
- Cuando llegue un escritor, bloquea el mutex de la base de datos.
- La escritura se bloqueará hasta que todos los lectores hayan terminado.
- Problema: ¿qué pasa si los lectores siguen llegando indefinidamente?

Ver `readers_writers1.c`

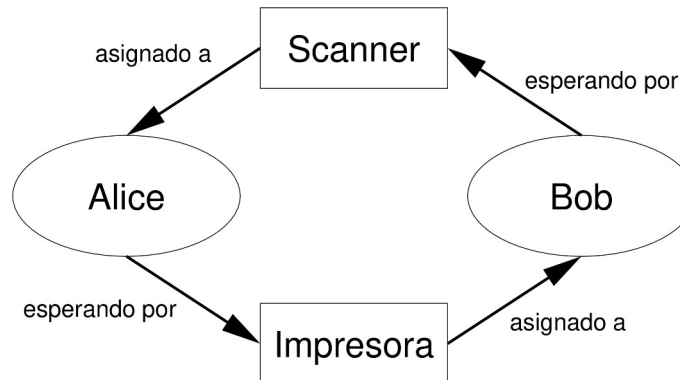


Lecciones aprendidas

- Los mutex simples no son suficientes; a menudo necesitamos más de uno.
- Los algoritmos complejos son difíciles de escribir y de entender. ¿Son correctos?
- Defina cuánta "justicia" y "eficiencia" necesita; factores como la inanición y los adelantamientos indefinidos pueden ser los verdaderos problemas.
- Esté atento a los interbloqueos.

Interbloqueos — El abrazo mortal

- Alice ha tomado el escáner y quiere usar la impresora.
- Bob ha tomado la impresora y quiere usar el escáner.
- Ninguno puede avanzar hasta que el otro lo haga.



Interbloqueos (Deadlocks)

- En un entorno de multiprogramación varios procesos compitiendo por un número finito de recursos.
- Un proceso solicita recursos, si los recursos no están disponibles en ese momento, el proceso se bloquea
- A veces, un proceso bloqueado nunca más puede cambiar de estado, porque los recursos que ha solicitado están en manos de otros procesos, los cuales están esperando que el proceso bloqueado realice una determinada acción.
 - Esta situación se llama un interbloqueo (Deadlock).

Historia

- Se estudió mucho en los inicios de los sistemas operativos.
- Apreciado por los teóricos.
- Hoy en día, es menos un problema del sistema operativo que de las aplicaciones, especialmente con las bases de datos.
- Hay técnicas generalmente aplicables.



Recurso

Cualquier cosa que en un instante determinado de tiempo sólo puede ser utilizada por un solo hilo/proceso.

- Dispositivos físicos
- Registros de bases de datos, archivos en disco
- Elementos compartibles como la memoria principal
- Altamente dependientes del sistema

Uso de recursos



Un proceso/hilo utiliza un recurso con la siguiente secuencia:

- 1) **Solicitud** del recurso. Si falla debe esperar con espera activa o bloqueo.
- 2) **Uso.**
- 3) **Liberación.**

Los problemas aparecen cuando hay dos o más recursos en juego.

Adquisición de recursos - caso un solo recurso

```
typedef int semaphore;  
semaphore resource1;  
  
void processA(void)  
{  
    down(&resource1);  
    use_resource1( );  
    up(&resource1);  
}
```

Adquisición de recursos – caso dos recursos

```
typedef int semaphore;  
semaphore resource1;  
semaphore resource2;  
  
void processA(void) {  
    down(&resource1);  
    down(&resource2);  
    use_both_resources( );  
    up(&resource2);  
    up(&resource1);  
}
```

Código libre de bloqueos

```
typedef int semaphore;

semaphore resource1;
semaphore resource2;

void processA(void) {
    down(&resource1);
    down(&resource2);
    use_both_resources();
    up(&resource2);
    up(&resource1);
}

void processB(void) {
    down(&resource1);
    down(&resource2);
    use_both_resources();
    up(&resource2);
    up(&resource1);
}
```

Código con bloqueo potencial

```
typedef int semaphore;

semaphore resource1;
semaphore resource2;

void processA(void) {
    down(&resource1);
    down(&resource2);
    use_both_resources();
    up(&resource2);
    up(&resource1);
}

void processB(void) {
    down(&resource2);
    down(&resource1);
    use_both_resources();
    up(&resource2);
    up(&resource1);
}
```

Comparación

Libre de bloqueos

```
process_A() {  
    down(&resource_1);  
    down(&resource_2);  
    use_resources();  
    up(&resource_1);  
    up(&resource_2);  
}  
  
process_B() {  
    down(&resource_1);  
    down(&resource_2);  
    use_resources();  
    up(&resource_1);  
    up(&resource_2);  
}
```

Con bloqueo potencial

```
process_A() {  
    down(&resource_1);  
    down(&resource_2);  
    use_resources();  
    up(&resource_1);  
    up(&resource_2);  
}  
  
process_B() {  
    down(&resource_2);  
    down(&resource_1);  
    use_resources();  
    up(&resource_1);  
    up(&resource_2);  
}
```

Sistema bloqueado

- Un conjunto de procesos está bloqueado **si cada proceso del conjunto está esperando por un evento que solo otro proceso del conjunto puede provocarlo.**
 - Todos los procesos están esperando.
 - Ninguno de ellos causará ninguno de los eventos que podrían despertar a cualquiera de los otros miembros del conjunto.
 - Todos los procesos continúan esperando para siempre.
 - El evento que cada proceso está esperando es la liberación de algún recurso que actualmente posee otro miembro del conjunto.

Condiciones para el interbloqueo (Coffman, 1971)

- Para que se produzca un interbloqueo **se tienen que cumplir** estas cuatro condiciones.
 - 1) Exclusión mutua: Cada recurso está disponible o asignado a un solo proceso.
 - 2) Retención y espera: Los procesos que poseen recursos pueden solicitar más.
 - 3) No apropiación: No se pueden retirar recursos de un proceso involuntariamente.
 - 4) Espera circular: Debe haber una cadena de dos o más procesos esperando al siguiente miembro.

El secreto para prevenir bloqueos mutuos

- Consiste simplemente eliminar cualquiera de las cuatro precondiciones.
 - No interesa cual
 - Entonces el bloqueo no puede suceder
 - Problema solucionado!

Estrategias para enfrentar los interbloqueos

- Ignorar el problema: quizá no sea lo suficientemente grave/frecuente
- Prevenirlo evitando una de las cuatro condiciones
- Detectarlo y solucionarlo
- Evitarlo mediante una asignación cuidadosa de recursos



1: Ignorar el problema

- Quizás la probabilidad de un bloqueo sea baja.
- Ejemplo: en mi máquina mi usuario puede tener 500 procesos ejecutándose y 1024 archivos abiertos como máximo.
- Supongamos que cada proceso que quisiera abrir un archivo y no pudiera, se quedaría en reposo un rato y lo volvería a intentar.
- Es fácil ver que un bloqueo podría ocurrir si muchos procesos abren muchos archivos.
- Siendo realistas, simplemente no sucede.
- El problema es evitable, pero solo a un alto costo.

Límites en linux
ulimit -a

Ignorando los bloqueos

- Los interbloqueos no siempre son malos
 - algunas veces podemos convivir con la posibilidad que se produzcan
 - cuando algo se cuelga, simplemente lo reiniciamos.
 - adecuado para muchas aplicaciones
- pero desafortunadamente, algunas veces no es posible.
 - dentro del sistema operativo, bases de datos, sistemas transaccionales.

2: Prevenir el problema

- La prevención se basa en:
 - Los interbloqueos son imposibles si podemos asegurar que una de las condiciones nunca se cumpla

Prevención: eliminando la exclusión mutua

- **Exclusión mutua:** recursos mantenidos por un procesos no pueden ser usado simultáneamente por otro.
 - Esto es difícil de evitar, normalmente la exclusión mutua se necesita o no.
 - Ej: Si todos los procesos acceden a un archivo en modo de lectura solamente se puede permitir que lo abran simultáneamente.
- La exclusión mutua puede ser evitada haciendo el uso del recurso "atómico"
 - Usar el recurso en una ráfaga de acceso exclusivo
 - Spooling.

Prevención: sacarle recursos

- Normalmente los recursos no se puede sacar: los procesos se quedan con los recursos hasta que ellos mismos los liberan.
- Es posible que se pueda eliminar con alguna clase de política.
 - Ej: si un proceso de alta prioridad quiere un recurso, lo obtiene y el recurso es retirado del proceso de baja prioridad.
- En la práctica no es fácil implementarlo
 - Cuando se saca un recurso la recuperación suele ser complicada.

Prevención: eliminando la retención y espera

- Retención y espera: se pueden acumular recursos, me bloqueo esperando mientras mantengo otros.
- Puede ser eliminada requiriendo que todo el conjunto requerido de recursos sea pedido de una sola vez.
 - Debe liberar todo para pedir un nuevo conjunto
- Ineficiente



Eliminación de retención y espera: Lockeo en dos fases

- Los procesos manejan los recursos en dos fases:
 - Fase 1: lockeo
 - intentan adquirir los locks
 - si cualquier intento falla, liberar todo y comenzar nuevamente.
 - Fase 2: uso
 - usar cualquier recurso solamente después que se adquirieron todos los locks
 - liberar los locks cuando se termina
- Seguro, pero ineficiente para muchas aplicaciones
 - Presenta problemas similares a los spin locks
 - Usado en bases de datos

Prevención: eliminando la espera circular

- **Espera circular:** cadena circular de procesos, cada uno esperando por un recurso que tiene el próximo proceso en la cadena.
 - Debe encontrarse la manera de organizar los pedidos tal que se rompan los ciclos.
- Idea central: **asignación jerárquica.**
 - numerar los recursos; solo permitir obtener recursos con numeración más alta que la que se tiene.
 - requiere encontrar la manera de numerar los recursos.

Asignación jerárquica en los filósofos

- La solución obvia es representar cada tenedor por un semáforo:
semaphore chopstick[N] DOWN antes de tomarlo y UP después de usarlo.

```
/* philosopher i loop */  
while (TRUE) {  
    DOWN(chopstick[i]);  
    DOWN(chopstick[(i+1) mod N]);  
    eat();  
    UP(chopstick[i]);  
    UP(chopstick[(i+1) mod N]);  
    think();  
}
```

- Si a este código le agregamos la condición que los semáforos se adquieran en orden numérico
 - Solucionamos los bloqueos (pero puede haber inanición)